《A Soft Sensor for Multirate Quality Variables Based on MC-CNN》方法报告

2026-03-27

目录

1 文档目标 **2**

2 方法总览 **2**

2.1 一句话概括 2

2.2 它解决的是什么问题 2

2.3 与普通 MIMO-CNN 的区别 3

3 符号、数据形式与张量形状 **3**

3.1 基础符号 3

3.2 标准化和空白值处理后的原始矩阵 3

3.3 多时间通道窗口化 4

3.4 一个非常重要的直觉 4

4 MC-CNN 的整体结构 **4**

4.1 结构分成两部分 4

4.2 共享网络的角色 5

4.3 并行预测网络的角色 5

5 逐层张量数据流分析 **5**

5.1 先说一个关键推断 5

5.2 共享网络：一般形状 5

5.3 TE 工况的具体形状 6

5.4 储能数据的具体形状 7

5.5 并行预测网络的形状 7

5.6 共享输出和最终输出不是一回事 7

6 卷积层到底在做什么 **8**

6.1 卷积不是直接沿时间轴滑动 8

6.2 为什么这样仍然能学习时滞关系 8

6.3 没有池化层的含义 9

7 修改版 BP 的数学本质 **9**
7.1 论文写法 9
7.2 更清晰的张量化改写 9
7.3 为什么空白值不会参与反向传播 10
7.4 它和 dropout 的区别 10

8 为什么多时间通道能“重复利用”稀疏标签 **10**
8.1 论文里的直觉 10
8.2 精确地写出来 10
8.3 这为什么有用 11

9 训练流程与推理流程 **11**
9.1 论文描述的训练流程 11
9.2 共享网络预训练阶段的数据流 11
9.3 并行预测头训练阶段的数据流 11
9.4 在线推理的数据流 12

10 实例化到论文中的两个实验 **12**
10.1 TE 过程 12
10.2 储能数据 13

11 从机器学习原理看 MC-CNN **13**
11.1 它本质上是在做什么 13
11.2 为什么共享网络有效 13
11.3 为什么适合多采样率质量变量 13

12 方法的优点、边界与实现注意点 **14**
12.1 优点 14
12.2 阅读论文时容易误解的地方 14
12.3 方法边界 14

13 结论 **14**

1 文档目标

本文围绕 Song 等人在 2025 年发表的 MC-CNN（Multitemporal Channel CNN）方法，给出一份面向实现和理解的中文技术报告。报告重点回答以下问题：

- 论文到底提出了什么方法，它想解决什么问题；
- 原始输入、训练标签、最终输出各自是什么，形状如何；
- 共享网络和并行预测网络的逐层数据流是什么样子；
- 论文中的“修改版 BP”在数学上等价于什么；
- 多时间通道为什么能提高稀疏质量变量的利用率。

文中会尽量保留论文原始记号，但会同时改写成更适合深度学习实现的张量视角。需要先说明一点：论文在网络结构部分给出了层维度，但没有像现代深度学习论文那样把每一层的张量轴定义得非常显式，因此本文对**卷积轴方向和若干中间张量形状**做了工程化推断。这些推断是有依据的，且能与论文给出的层维度完全对齐。

2 方法总览

2.1 一句话概括

MC-CNN 的核心思想是：**把最近 M 个时刻的过程变量堆叠成 M 个时间通道，用共享网络学习“过程变量到多时刻质量变量窗口”的映射，再用并行子网络把共享特征变成每个质量变量当前时刻的预测值**。同时，通过掩码式反向传播，让未采样时刻产生的空白标签不参与梯度计算。

2.2 它解决的是什么问题

工业软测量里经常出现如下情况：

- 过程变量 X 可以高频在线采样；
- 质量变量 Y 只能低频采样，甚至不同质量变量采样周期还不同；
- 目标却是希望在每个时刻都输出质量变量的预测值。

如果直接用普通的 MIMO 回归模型，会遇到两个问题：

- 大量时刻没有真实质量标签，监督信号过于稀疏；
- 只看“当前时刻”输入，难以学习过程变量和质量变量之间的时滞关系。

MC-CNN 针对这两个问题同时设计了两个机制：

- **多时间通道**：把连续 M 个时刻的过程变量打包输入，从而显式建模时序依赖；
- **修改版 BP**：对空白标签不回传梯度，使模型可以直接在带缺测标签的窗口上训练。

2.3 与普通 MIMO-CNN 的区别

论文里专门构造了一个对照模型 MIMO-CNN。两者最本质的区别不是“有没有 CNN”，而是**最后一层监督方式不同**：

- MIMO-CNN 只用单时刻标签训练，本质上仍然主要学习“当前输入到当前输出”；
- MC-CNN 用 M 个时刻组成的质量变量窗口 D_y 监督共享网络，因此共享特征会被迫对多时刻标签同时有解释力。

这一点是整篇论文最值得抓住的设计核心。

3 符号、数据形式与张量形状

3.1 基础符号

设：

- T 为总采样时刻数；
- m 为过程变量个数；
- n 为质量变量个数；
- M 为多时间通道数，也就是时间窗口长度；
- $x_t \in \mathbb{R}^m$ 为第 t 个时刻的过程变量向量；
- $y_t \in \mathbb{R}^n$ 为第 t 个时刻的质量变量向量。

由于质量变量是多采样率的，所以 y_t 的某些分量在很多时刻并不存在真实观测值。论文把这些位置视为 blank value（空白值）。

3.2 标准化和空白值处理后的原始矩阵

论文在预处理后得到

$$X^{nb} = [x_1, x_2, \ldots, x_T]^\top \in \mathbb{R}^{T\times m}, \tag{1}$$

$$Y^{nb} = [y_1, y_2, \ldots, y_T]^\top \in \mathbb{R}^{T\times n}. \tag{2}$$

这里上标 nb 可以理解为“完成归一化并处理 blank 值之后”的数据形式。严格来说：

- X^{nb} 一般是完整观测的；
- Y^{nb} 中未采样位置会被填上一个特殊值，训练时再通过修改版 BP 忽略掉。

从现代实现角度看，更推荐把这个“特殊值”显式写成掩码矩阵

$$R \in \{0,1\}^{T\times n}, \tag{3}$$

其中 $R_{t,q} = 1$ 表示第 t 个时刻第 q 个质量变量有真实观测。

3.3 多时间通道窗口化

论文把长度为 M 的滑动窗口拼接成三维张量。若记

$$S = T - M + 1, \tag{4}$$

则有

$$D_x = \begin{bmatrix} [x_1, x_2, \ldots, x_M]^\top \\ [x_2, x_3, \ldots, x_{M+1}]^\top \\ \vdots \\ [x_{T-M+1}, x_{T-M+2}, \ldots, x_T]^\top \end{bmatrix} \in \mathbb{R}^{S\times M\times m}, \tag{5}$$

$$D_y = \begin{bmatrix} [y_1, y_2, \ldots, y_M]^\top \\ [y_2, y_3, \ldots, y_{M+1}]^\top \\ \vdots \\ [y_{T-M+1}, y_{T-M+2}, \ldots, y_T]^\top \end{bmatrix} \in \mathbb{R}^{S\times M\times n}. \tag{6}$$

因此，**单个训练样本**的形状是：

$$d_x^{(s)} \in \mathbb{R}^{M\times m}, \qquad d_y^{(s)} \in \mathbb{R}^{M\times n}. \tag{7}$$

若按 batch 训练，batch size 为 B，则输入输出张量分别是：

$$\mathcal{X}_{\text{batch}} \in \mathbb{R}^{B\times M\times m}, \qquad \mathcal{Y}_{\text{batch}} \in \mathbb{R}^{B\times M\times n}. \tag{8}$$

3.4 一个非常重要的直觉

窗口 D_x 与 D_y 的构造意味着：**模型训练时不是在看一个时刻，而是在看一段长度为 M 的局部时间片段**。这让共享网络有机会学习：

- 哪个质量变量更依赖最近时刻的过程变量；
- 哪个质量变量更依赖更早的滞后信息；
- 多个质量变量之间在同一时间片上的互补模式。

4 MC-CNN 的整体结构

4.1 结构分成两部分

论文把 MC-CNN 明确分成两块：

- **共享网络（shared network）**：输入是 D_x，监督标签是 D_y，用于提取共享特征 $h(k)$；
- **并行预测网络（parallel prediction network）**：每个质量变量各有一个子网络，以共享特征 $h(k)$ 为输入，输出该质量变量当前时刻的预测值。

图示上可以把它理解为：

$$D_x \longrightarrow \text{Shared Network} \longrightarrow h(k) \longrightarrow \begin{cases} \text{Head}_1 \to \hat{y}_1(k) \\ \text{Head}_2 \to \hat{y}_2(k) \\ \vdots \\ \text{Head}_n \to \hat{y}_n(k) \end{cases} \tag{9}$$

同时，共享网络内部还会生成一个窗口级输出 $\hat{D}_y \in \mathbb{R}^{M\times n}$，用于第一阶段的预训练。

4.2 共享网络的角色

共享网络的目标不是直接给出最终在线预测，而是先回答一个更容易学的问题：

给定最近 M 个时刻的过程变量窗口，能否恢复同一窗口里的质量变量模式？

这样做有两个好处：

- 共享层能被多时刻监督，得到更稳定的时序特征；
- 稀疏标签会在多个窗口位置重复出现，提高有效监督次数。

4.3 并行预测网络的角色

并行预测网络才是最终软测量输出层。它把共享特征 $h(k)$ 投影为当前时刻各质量变量的预测值：

$$\hat{y}(k) = [\hat{y}_1(k), \hat{y}_2(k), \ldots, \hat{y}_n(k)]^\top \in \mathbb{R}^n. \tag{10}$$

因此，从任务分工上看：

- 共享网络负责“抽取多时刻、多变量共同有用的表示”；
- 每个预测头负责“把共享表示变成某个具体质量变量的标量回归结果”。

5 逐层张量数据流分析

5.1 先说一个关键推断

论文给出的 TE 工况共享网络层维度为：

$$[[5,33]-[5,40]-[3,38]-[36]-[25]-[5,2]]. \tag{11}$$

这组维度几乎唯一地说明了如下张量解释：

- 第一维的 5 不是 batch，也不是特征数，而是**多时间通道数** M；
- 第一层全连接是对每个时间通道上的 33 维过程变量向量独立做线性变换，得到 40 维；
- 之后的卷积层把 $M = 5$ 当作输入通道数，把投影后的特征轴长度 40 当作卷积的一维长度。

也就是说，**这不是“沿时间轴滑动的时域卷积”，而是“时间滞后作为通道，卷积沿投影特征轴滑动”**。时间信息是通过通道混合进入卷积的，这一点非常重要。

5.2 共享网络：一般形状

设一个 batch 的输入为

$$X_b \in \mathbb{R}^{B\times M\times m}. \tag{12}$$

共享网络可以写成以下数据流：

步骤 1：逐通道全连接投影

$$H^{(0)} = \phi(X_b W_1 + b_1), \tag{13}$$

其中 ϕ 表示 SELU 激活。若投影后宽度为 d_1，则

$$H^{(0)} \in \mathbb{R}^{B \times M \times d_1}. \tag{14}$$

步骤 2：第一层 1D 卷积

$$H^{(1)} = \phi(\mathrm{Conv1d}(H^{(0)})), \tag{15}$$

若输出通道数为 c_1、卷积核宽度为 k_1、步长为 1，则

$$H^{(1)} \in \mathbb{R}^{B \times c_1 \times (d_1 - k_1 + 1)}. \tag{16}$$

步骤 3：第二层 1D 卷积

$$H^{(2)} = \phi(\mathrm{Conv1d}(H^{(1)})), \tag{17}$$

若第二层输出通道数为 1、卷积核宽度为 k_2，则

$$H^{(2)} \in \mathbb{R}^{B \times 1 \times (d_1 - k_1 - k_2 + 2)}. \tag{18}$$

步骤 4：展平

$$z = \mathrm{Flatten}(H^{(2)}) \in \mathbb{R}^{B \times (d_1 - k_1 - k_2 + 2)}. \tag{19}$$

步骤 5：共享隐特征

$$h = \phi(z W_h + b_h) \in \mathbb{R}^{B \times d_h}. \tag{20}$$

步骤 6：窗口级输出

$$\hat{D}_y = \mathrm{reshape}(h W_o + b_o) \in \mathbb{R}^{B \times M \times n}. \tag{21}$$

5.3 TE 工况的具体形状

在 TE 实验中，论文给出：

- $M = 5$；
- $m = 33$；
- $n = 2$；
- 共享网络维度为 $[[5, 33] - [5, 40] - [3, 38] - [36] - [25] - [5, 2]]$。

于是 batch 级形状可以明确写成：

$$X_b \in \mathbb{R}^{B \times 5 \times 33}, \tag{22}$$

$$H^{(0)} \in \mathbb{R}^{B \times 5 \times 40}, \tag{23}$$

$$H^{(1)} \in \mathbb{R}^{B \times 3 \times 38}, \tag{24}$$

$$H^{(2)} \in \mathbb{R}^{B \times 1 \times 36}, \tag{25}$$

$$z \in \mathbb{R}^{B \times 36}, \tag{26}$$

$$h \in \mathbb{R}^{B \times 25}, \tag{27}$$

$$\hat{D}_y \in \mathbb{R}^{B \times 5 \times 2}. \tag{28}$$

这里的两个卷积核宽度都可由维度反推出为 3，因为：

$$40 \xrightarrow{k=3} 38 \xrightarrow{k=3} 36. \tag{29}$$

5.4 储能数据的具体形状

在储能数据实验中，论文给出的共享网络维度为

$$[[5,640]-[5,320]-[5,160]-[5,80]-[5,40]-[3,38]-[36]-[25]-[5,2]]. \tag{30}$$

因此对应的数据流是：

$$X_b \in \mathbb{R}^{B\times 5\times 640}, \tag{31}$$
$$H_1^{(0)} \in \mathbb{R}^{B\times 5\times 320}, \tag{32}$$
$$H_2^{(0)} \in \mathbb{R}^{B\times 5\times 160}, \tag{33}$$
$$H_3^{(0)} \in \mathbb{R}^{B\times 5\times 80}, \tag{34}$$
$$H_4^{(0)} \in \mathbb{R}^{B\times 5\times 40}, \tag{35}$$
$$H^{(1)} \in \mathbb{R}^{B\times 3\times 38}, \tag{36}$$
$$H^{(2)} \in \mathbb{R}^{B\times 1\times 36}, \tag{37}$$
$$z \in \mathbb{R}^{B\times 36}, \tag{38}$$
$$h \in \mathbb{R}^{B\times 25}, \tag{39}$$
$$\hat{D}_y \in \mathbb{R}^{B\times 5\times 2}. \tag{40}$$

可以看出，两组实验虽然输入维度差别很大，但作者都先把每个时间通道上的高维过程变量压到 40 维，再进入相同的卷积–展平–隐层结构。

5.5 并行预测网络的形状

论文给出的每个质量变量预测头结构是：

$$[25-5-1]. \tag{41}$$

这意味着第 q 个质量变量的预测头可以写成

$$\hat{y}_q = W_{q,2}\,\phi(W_{q,1}h + b_{q,1}) + b_{q,2}, \tag{42}$$

其中

$$h \in \mathbb{R}^{B\times 25}, \qquad \hat{y}_q \in \mathbb{R}^{B\times 1}. \tag{43}$$

把所有预测头拼起来之后：

$$\hat{Y}_{\text{latest}} \in \mathbb{R}^{B\times n}. \tag{44}$$

如果是论文中的两个质量变量案例，则

$$\hat{Y}_{\text{latest}} \in \mathbb{R}^{B\times 2}. \tag{45}$$

5.6 共享输出和最终输出不是一回事

这是阅读论文时最容易混淆的一点。

共享网络输出

$$\hat{D}_y \in \mathbb{R}^{B\times M\times n} \tag{46}$$

是窗口级输出，用来做第一阶段训练监督。

并行预测网络输出

$$\hat{Y}_{\text{latest}} \in \mathbb{R}^{B\times n} \tag{47}$$

才是最终用于在线软测量的当前时刻预测。

所以更准确地说，MC-CNN 有**两个输出层次**：

- 一个是“恢复整个质量窗口”的辅助输出；
- 一个是“预测当前时刻各质量变量”的最终输出。

6 卷积层到底在做什么

6.1 卷积不是直接沿时间轴滑动

如果采用 PyTorch/深度学习框架的标准约定，1D 卷积的输入通常是

$$\mathbb{R}^{B\times C\times L}, \tag{48}$$

其中 C 是通道数，L 是卷积长度。

在本文结构里，最自然的解释是：

- $C = M$，即 M 个时间通道；
- $L = d_1$，即每个时间通道经全连接投影后的特征长度。

因此第一层卷积的一个输出元素可以写成

$$H^{(1)}_{b,r,p} = \phi\left(\sum_{c=1}^{M}\sum_{\delta=0}^{k_1-1} K^{(1)}_{r,c,\delta}\, H^{(0)}_{b,c,p+\delta} + b^{(1)}_r\right), \tag{49}$$

其中：

- r 是输出卷积核编号；
- c 遍历所有时间通道；
- δ 遍历卷积核局部感受野。

这个式子说明：**每个卷积核会同时混合所有时间通道的信息**。因此，虽然卷积核不是沿“时间轴”平移，但时间滞后信息已经通过通道维进入了卷积运算。

6.2 为什么这样仍然能学习时滞关系

原因在于：第 $1,\ldots,M$ 个通道本身就分别代表不同的相对时间位置。比如当 $M = 5$ 时：

$$d_x^{(s)} = \begin{bmatrix} x_s \\ x_{s+1} \\ x_{s+2} \\ x_{s+3} \\ x_{s+4} \end{bmatrix}. \tag{50}$$

模型看到的并不是一组无序向量，而是一个带有固定相对顺序的通道集合。不同通道权重不同，就意味着模型能区分：

- 当前质量值更依赖 x_t；
- 还是更依赖 x_{t-1}、x_{t-2}；
- 甚至不同质量变量依赖的滞后步长不同。

6.3 没有池化层的含义

论文明确说 MC-CNN 不使用 pooling 层，因为作者担心池化会滤掉“次要特征”，而这些次要特征对软测量回归可能仍然有用。这个选择说明作者更关心**保留细粒度回归信息**，而不是像图像分类那样追求平移不变性。

7 修改版 BP 的数学本质

7.1 论文写法

论文定义了一个函数

$$g(x-y)=\begin{cases} x-y, & y\in y_r, \\ 0, & y\notin y_r, \end{cases} \tag{51}$$

其中 y_r 表示真实采样值集合。然后把共享网络的损失写成

$$J_s=\frac{1}{2NZ}\sum_{i=1}^{N}\sum_{j=1}^{M}\left\{g(\tilde{y}(i,j)-y(i,j))\right\}^2, \tag{52}$$

并把预测头损失写成

$$J_p=\frac{1}{2Z}\sum_{i=1}^{N}\left\{g(\tilde{y}(i)-y(i))\right\}^2. \tag{53}$$

这里 Z 是标签里真实值的个数。

7.2 更清晰的张量化改写

更现代、也更容易实现的写法是直接引入 mask。设

$$Q\in\{0,1\}^{B\times M\times n} \tag{54}$$

表示窗口标签是否真实可用，那么共享网络损失可以写成

$$\mathcal{L}_{\text{shared}}=\frac{1}{2\sum Q}\left\|Q\odot(\hat{D}_y-D_y)\right\|_F^2. \tag{55}$$

若定义当前时刻标签掩码

$$q^{\text{latest}}\in\{0,1\}^{B\times n}, \tag{56}$$

则并行预测头的损失为

$$\mathcal{L}_{\text{head}}=\frac{1}{2\sum q^{\text{latest}}}\left\|q^{\text{latest}}\odot(\hat{Y}_{\text{latest}}-Y_{\text{latest}})\right\|_F^2. \tag{57}$$

这里 $\odot$ 表示按元素乘法。

7.3 为什么空白值不会参与反向传播

对共享损失中的任意一个位置 (b, c, q)，有

$$\frac{\partial \mathcal{L}_{\text{shared}}}{\partial \hat{D}_{y,b,c,q}} = \frac{Q_{b,c,q}}{\sum Q} \left(\hat{D}_{y,b,c,q} - D_{y,b,c,q} \right). \tag{58}$$

如果该位置没有真实观测，那么 $Q_{b,c,q} = 0$，于是

$$\frac{\partial \mathcal{L}_{\text{shared}}}{\partial \hat{D}_{y,b,c,q}} = 0. \tag{59}$$

这就说明：**未采样位置对损失和梯度都没有贡献**。论文所谓的“修改版 BP”，本质上就是今天常说的 **masked regression loss**。

7.4 它和 dropout 的区别

论文特别强调这和 dropout 不是一回事。这个判断是正确的：

- dropout 是一种随机正则化，目的是防止过拟合；
- MC-CNN 这里屏蔽的是**数据中本来就不存在真实标签的位置**，目的是避免错误监督。

前者是人为随机丢弃，后者是对缺失监督做严格掩码，二者功能和数学语义都不同。

8 为什么多时间通道能“重复利用”稀疏标签

8.1 论文里的直觉

论文举例说，当 $M = 5$ 时，一个真实标签最多可以放到输出层的 5 个不同位置，因此“一个真实值可以被利用五次”。这句话的意思不是复制标签，而是指**同一个真实采样时刻会出现在多个滑动窗口中**。

8.2 精确地写出来

设第 t 个时刻第 q 个质量变量有真实采样值 $y_{t,q}$。它会出现在所有满足

$$s \le t \le s + M - 1 \tag{60}$$

的窗口中，也就是

$$s \in [t - M + 1,\ t] \tag{61}$$

与合法窗口起点区间 $[1, S]$ 的交集内。

因此，这个真实值参与训练的次数为

$$c_t = \left| [t - M + 1,\ t] \cap [1, S] \right|. \tag{62}$$

对中间时刻的样本，通常有

$$c_t = M. \tag{63}$$

边界附近的样本参与次数会少一些，但**绝大多数内部样本都能被重复利用 M 次**。

8.3 这为什么有用

因为每次它出现时，对应的输入上下文都不同。比如同一个真实值 y_t，可能分别与如下过程窗口配对：

$$[x_{t-4}, x_{t-3}, x_{t-2}, x_{t-1}, x_t], \tag{64}$$

$$[x_{t-3}, x_{t-2}, x_{t-1}, x_t, x_{t+1}], \tag{65}$$

$$\dots \tag{66}$$

于是模型不只学到“当前过程状态和当前质量值的关系”，还学到“不同相对滞后位置下，哪些过程模式更能解释这个质量值”。这正是 MC-CNN 想捕捉的 temporal correlations。

9 训练流程与推理流程

9.1 论文描述的训练流程

根据论文正文，训练大致是一个两阶段过程：

1. 收集过程变量和质量变量，划分训练集与测试集；
2. 对数据做 Z-score 标准化；
3. 对未采样质量变量填入特殊值，并在训练中通过修改版 BP 忽略；
4. 把 X 和 Y 窗口化为 D_x 与 D_y；
5. 先用 (D_x, D_y) 预训练共享网络，提取共享特征 $h(k)$；
6. 再用共享特征 $h(k)$ 和当前时刻质量标签训练并行预测头；
7. 推理时，把最近 M 个时刻的过程变量输入模型，输出当前时刻各质量变量预测值。

9.2 共享网络预训练阶段的数据流

在 batch 级别，这一阶段的输入输出可以总结为：

$$\mathbb{R}^{B\times M\times m} \longrightarrow \mathbb{R}^{B\times M\times n}. \tag{67}$$

也就是

$$X_b \xrightarrow{\text{shared network}} \hat{D}_y \xrightarrow{\text{masked loss}} D_y. \tag{68}$$

这一步训练出来的重点不是最终输出层，而是中间特征

$$h \in \mathbb{R}^{B\times 25}. \tag{69}$$

9.3 并行预测头训练阶段的数据流

第二阶段对每个质量变量单独训练一个头。若批量当前时刻标签记为

$$Y_{\text{latest}} \in \mathbb{R}^{B\times n}, \tag{70}$$

则这一阶段是

$$h \in \mathbb{R}^{B\times 25} \longrightarrow \hat{Y}_{\text{latest}} \in \mathbb{R}^{B\times n} \xrightarrow{\text{masked loss}} Y_{\text{latest}}. \tag{71}$$

一个实现细节 论文文字表述更像是“先训练共享网络，再训练预测头”。它没有明确说明第二阶段是否继续联合更新共享网络参数。最保守的理解是：

- 第一阶段学习共享特征；
- 第二阶段把共享特征当作固定输入训练各预测头。

如果采用端到端联合微调，那属于与论文主线一致、但略偏工程扩展的实现方式。

9.4 在线推理的数据流

在线推理时，不需要真实质量标签。只要最新的过程变量窗口可得，即可执行：

$$[x_{t-M+1}, \ldots, x_t] \longrightarrow h(t) \longrightarrow [\hat{y}_1(t), \ldots, \hat{y}_n(t)]. \tag{72}$$

因此最终软测量器的输入输出关系是：

$$\mathbb{R}^{M\times m} \to \mathbb{R}^n. \tag{73}$$

这也解释了为什么论文说 MC-CNN 可以在**高频**上输出质量变量预测值：即使真实质量变量只有低频采样，模型也可以利用每个时刻都有的过程变量持续推断。

10 实例化到论文中的两个实验

10.1 TE 过程

论文中 TE 过程的设置为：

- 过程变量数 $m = 33$；
- 质量变量数 $n = 2$；
- 多时间通道数 $M = 5$；
- 共享网络维度：$[[5, 33] - [5, 40] - [3, 38] - [36] - [25] - [5, 2]]$；
- 每个质量变量预测头：$[25 - 5 - 1]$；
- 超参数表给出：预训练学习率 0.01、epoch 8、batch size 32；微调学习率 0.01、epoch 6、batch size 32。

因此 TE 过程里，一个 batch 的典型前向形状是

$$B \times 5 \times 33 \to B \times 5 \times 40 \to B \times 3 \times 38 \to B \times 1 \times 36 \to B \times 36 \to B \times 25 \to B \times 5 \times 2, \tag{74}$$

随后再由并行头得到

$$B \times 25 \to B \times 2. \tag{75}$$

10.2 储能数据

论文中储能数据的设置为：

- 过程变量数 $m = 640$；
- 质量变量数 $n = 2$；
- 多时间通道数 $M = 5$；
- 共享网络维度：$[[5, 640] - [5, 320] - [5, 160] - [5, 80] - [5, 40] - [3, 38] - [36] - [25] - [5, 2]]$；
- 每个质量变量预测头：$[25 - 5 - 1]$；
- 超参数表给出：预训练学习率 0.01、epoch 6、batch size 32；微调学习率 0.01、epoch 6、batch size 32。

这里的主要区别只是前端增加了若干层逐通道全连接，用来把 640 维高维输入逐步压缩到 40 维。后面的卷积、展平和隐层结构与 TE 设置保持一致。

11 从机器学习原理看 MC-CNN

11.1 它本质上是在做什么

如果不拘泥于论文术语，MC-CNN 可以理解为下面这个组合：

- **滑动窗口监督学习**：把时序问题转成局部窗口回归问题；
- **多任务学习**：共享网络同时服务于多个质量变量；
- **带掩码的缺失标签回归**：未观测标签不计入损失；
- **滞后建模**：最近 M 个时刻被显式编码到输入通道中。

11.2 为什么共享网络有效

共享网络被要求同时解释：

- 同一个质量变量在不同时间位置上的值；
- 不同质量变量之间的联合模式。

因此学到的隐特征 $h(k)$ 不太可能只对单个输出有用，它更像是一个**对局部过程状态的多变量时序摘要**。这也是后续每个质量变量预测头都能复用它的原因。

11.3 为什么适合多采样率质量变量

对多采样率任务，最麻烦的是监督很稀疏。MC-CNN 的做法不是先插值补齐所有质量变量，而是：

- 保留真实采样时刻；
- 用 mask 屏蔽未采样位置；
- 通过滑动窗口让真实采样值出现在多个训练样本中。

这比直接插值更稳妥，因为它没有把“伪标签”当成真值强行喂给网络。

12 方法的优点、边界与实现注意点

12.1 优点

- 能同时处理多个质量变量；
- 能处理不同质量变量采样周期不同的情况；
- 通过窗口监督提高低频标签利用率；
- 结构较小，作者强调参数量和训练时间都比较可控；
- 推理阶段只依赖过程变量窗口，适合在线预测。

12.2 阅读论文时容易误解的地方

- “多时间通道 CNN”不等价于标准 TCN。它不是在时间轴上做因果卷积，而是把时间滞后编码到通道中。
- [5, 2] 这个输出维度是共享网络窗口输出，不是最终在线输出。最终在线输出是 [2]。
- 论文的“修改版 BP”本质上就是 masked MSE，只是用特殊值加条件函数的方式表达。

12.3 方法边界

- 采样间隔若非常不规则，固定长度窗口 M 的表达能力会受限；
- 质量变量极少、过程变量极多时，前端全连接层仍然可能有较大参数量；
- 该结构更像“滞后特征提取器”，并不像 Transformer 或 RNN 那样显式建模长程依赖；
- 论文把 M 当作重要超参数，选太小学不到足够时滞，选太大则训练开销上升。

13 结论

MC-CNN 的真正创新点不在于“把 CNN 用到软测量”本身，而在于它把**多时间通道**、**窗口级监督**、**并行多输出预测**、**以及掩码式反向传播**组合到了一起，专门应对“过程变量高频、质量变量低频且多采样率”的场景。

如果用一句更工程化的话来总结，它做的是：

> 先用最近 M 个时刻的过程变量去重建一个带缺测的质量变量窗口，从中学到共享时序特征；再把这个共享特征送入多个小回归头，输出每个质量变量当前时刻的高频预测值。

从张量角度看，MC-CNN 的关键数据流更准确地应写成

$$\mathbb{R}^{B\times M\times m} \to \mathbb{R}^{B\times 25} \to \begin{cases} \mathbb{R}^{B\times M\times n}, \\ \mathbb{R}^{B\times n}, \end{cases} \tag{76}$$

其中共享隐特征先由窗口输入得到，再分别通向窗口级监督输出和最终在线软测量输出。

从数学角度看，它的“修改版 BP”可以非常清楚地理解为：

> 对真实采样标签做均方误差，对空白标签位置乘零掩码，不让错误监督传播到网络参数。

这也是 MC-CNN 能在低频质量变量条件下仍然保持较好训练效果的关键所在。